iacc instituto profesional

EVALUACIÓN

Herramientas de QA
Semana 1

Amparo Valdivia
20, junio de 2025
Técnico en Análisis y Programación.

DESARROLLO:

Durante el desarrollo de un software para la gestión de historias clínicas, la precisión y seguridad de los datos médicos son aspectos fundamentales para garantizar la calidad del software, para llevar a cabo esta labor y asegurar la calidad en los procesos se hace verificando durante el desarrollo del sistema, el software e implementación desde etapas iniciales y el cumplimiento de las normas, los test, pruebas.

Debemos tener una hoja de ruta elaborada para poder comprender tanto los requisitos necesarios para realizar un buen desarrollo del sistema y una alta madurez en el proceso de calidad y validación final del proceso, esto puede lograrse realizando etapas, ya que solo así se puede lograr la calidad global del sistema, teniendo control de cada etapa:

- **Conocer el Contexto (Dominio y Propósito):** Entender la industria en este caso de salud, el tipo de software como “Historias Clínicas Electrónicas”, su criticidad (precisión, seguridad, privacidad, confidencialidad) y su uso, parámetros esenciales para lograr las funciones del software.
- **Identificar las Reglas Legales y Regulatorias (Obligatorias):** Leyes que el software debe cumplir por ley en Chile o el del país que corresponda, y cualquier otro relevante si es un producto internacional. Sin esto, el producto no puede operar legalmente.
- **Comprender los Requisitos del Cliente:** Los estándares de calidad o requisitos específicos del cliente pueden ser aún más altos que la regulación mínima, debido a esto es importante definir este requisito desde una etapa inicial manteniéndola constantemente verificada.

- **Investigar las Mejores Prácticas de la Industria (Normas como ISO/IEC 25000, 12207):** Estas son las guías de excelencia. Aunque no siempre sean legalmente obligatorias, adoptarlas eleva la calidad, la confianza y la eficiencia del proceso de desarrollo.
- **Idear un Plan Integrado (Plan de Calidad y Pruebas):** crear un plan estratégico que:
    - Defina los **objetivos de calidad** basados en las normas y requisitos.
    - Establezca la **estrategia de pruebas** para validar cada aspecto (funcionalidad, seguridad, privacidad, fiabilidad, rendimiento, etc.).
    - Asigne **recursos** (personas, herramientas, tiempo).
    - Defina los **criterios de entrada y salida** de cada fase de pruebas.
    - Identifique los **riesgos** y cómo mitigarlos, documentándolos para tener una reseña histórica tanto de los riesgos, las fallas y las pruebas resultantes positivas.
- **Hacer Pasos Organizados (Ciclo de Vida de Pruebas - STLC):** Dividir el plan en etapas claras, sistemáticas y verificar su cumplimiento, ya que no basta con definirlas sino también asegurar su implementación, así en el futuro y su constante uso, el equipo de trabajo logra hacer parte la estructura a su trabajo diario:
    - Análisis de requisitos.
    - Planificación.
    - Diseño de casos de prueba.
    - Configuración del entorno.
    - Ejecución.
    - Cierre. Cada etapa tiene sus entregables y validaciones.

- **Organizar las Pruebas (Diseño de Casos de Prueba):** Traducir los requisitos y las normas en pasos de prueba concretos y verificables, con resultados esperados. Por ejemplo, si una norma exige "autenticación fuerte", se diseñan casos de prueba para verificar contraseñas complejas, bloqueo de cuentas, multi-factor authentication, tratando de ponerse en todos los escenarios posibles.
- **Documentar Continuamente:** No solo al final. Cada paso, cada decisión, cada prueba ejecutada, cada defecto encontrado, cada lección aprendida debe quedar registrada. Esto es vital para la trazabilidad, la auditoría y la mejora futura, no olvidando poner reglas de documentación para que estas contengan información guardable y equivalente para realizar análisis y comparaciones en una base de datos acorde.

Ahora que he podido definir el ciclo de trabajo a seguir, profundizaré en cada punto para dar solución a la búsqueda de la calidad, describiendo procesos y subprocesos para definir los criterios de calidad correspondientes a cada proceso.

1.- Conocer el Contexto (Dominio y Propósito): Entender la industria en este caso de salud.

Cómo identificar las normativas vigentes de validación de calidad del software en empresas que entregan servicios de salud, para conocer el contexto a trabajar.

- Características distintivas.
- Requisitos de seguridad, **Seguridad del Paciente (Riesgo de Vida):** El software en salud maneja información que puede afectar directamente diagnósticos, tratamientos, dosificación de medicamentos, y decisiones clínicas. Un error de

software debido a una falta de calidad o incumplimiento normativo puede tener consecuencias graves, incluso mortales, para los pacientes.

- Privacidad, confidencialidad, **Precisión e Integridad de los Datos Médicos:** Las historias clínicas electrónicas requieren una precisión absoluta. Errores en la entrada, almacenamiento o recuperación de datos pueden llevar a diagnósticos erróneos o tratamientos inadecuados.
- Las normativas aseguran la fiabilidad y exactitud de estos datos. ISO/IEC 25000 exige que el software realice sus tareas con la precisión requerida. Esto significa que las funciones de gestión de historias clínicas (ej. cálculo de dosis, registro de signos vitales, alertas) deben ser numéricamente y lógicamente correctas, garantizando la fidelidad de los datos médicos.
- La **Fiabilidad** también contribuye, ya que asegura que el software funcione de manera consistente, evitando errores o fallos que puedan corromper o manipular datos precisos.

- **Privacidad y Confidencialidad de la Información Sensible:** Los datos de salud son altamente sensibles y están protegidos por leyes estrictas (como HIPAA en EE. UU. o GDPR en Europa, y normativas locales como en Chile). El incumplimiento de estas normas de privacidad y confidencialidad no solo genera pérdida de confianza, sino también graves sanciones legales y económicas para la empresa.
- **Cumplimiento Regulatorio y Legal:** El sector de la salud está fuertemente regulado. El incumplimiento de las normativas de validación de software puede resultar en multas elevadas, prohibición de operar, demandas legales y daño reputacional irreparable.

- **Gestión de Riesgos:** realizar un análisis exhaustivo de estas normativas, comprendiendo su alcance, las normativas proporcionan un marco para identificar, evaluar y mitigar los riesgos asociados con el desarrollo y uso del software en un entorno crítico.
- Objetivos, **Confianza y Credibilidad:** Para una empresa líder en tecnología del servicio de atención de la salud, el cumplimiento normativo es un pilar de su credibilidad y la confianza de sus clientes (hospitales, clínicas) y usuarios finales (profesionales de la salud, pacientes).
- Describir los criterios de evaluación.

2.- Identificar las Reglas Legales y Regulatorias (Obligatorias):

Además de las normas existentes en chile está regulado por el ministerio de salud La POLÍTICA DE INSTALACIÓN Y USO DE SOFTWARE Y PREVENCIÓN CONTRA CÓDIGO MALICIOSO INSTITUTO DE SALUD PÚBLICA DE CHILE, que indica en su Fecha de Emisión: 09/08/2019 Versión: 1 Fecha de actualización: 21/11/2019, aprobó el presente documento, teniendo en consideración la NCh-ISO 27001 Of 2013 y el Sistema de Gestión Integrado, bajo las normas ISO 9001, ISO IEC 17025, ISO 15189, ISO IEC 17043, ISO 17034, ISO Guide 35 y Norma Técnica 139/2012 de Buenas Prácticas de Laboratorio de la OMS.

Este documento expresa la acción del marco en que se debe trabajar tanto en la instalación como en la utilización de software en equipos y servidores de uso institucional, dentro de sus regulaciones es importante destacar, El Procedimiento de Gestión de Proyectos y Sistemas, PR-611.00-011; El Procedimiento de Monitoreo, Registro y Protección de Registro de Eventos, PR-611.00- 013; Preservación de la confidencialidad, integridad y disponibilidad de la información. (Ref ISO 27000:2018).

Ley 21541 modifica la normativa que indica para autorizar a los prestadores de salud a efectuar atenciones mediante telemedicina, el resguardo de los datos de los pacientes y el

acceso del paciente a comunicarse con el servicio de salud mediante un acceso único personal.

¿Cómo aplicar las normas para cada caso?

- Es una **planificación proactiva y una integración continua**.
- **Identificación:** Saber qué normas aplican a el proyecto y sus requisitos específicos.
- **Interpretación:** Entender qué significa cada norma en términos de requisitos funcionales y no funcionales del software.
- **Diseño de Pruebas:** Crear casos de prueba específicos que validen el cumplimiento de esas normas.
- **Integración:** Incorporar las directrices de las normas en cada etapa del ciclo de vida del desarrollo de software (diseño, codificación, pruebas).
- **Aplicar:** Las normativas de validación de calidad del software de manera efectiva en los proyectos de desarrollo.

Según el material de profundización del ramo, semana 1, La norma ISO/IEC 25000 donde cada característica principal se desglosa en sub características más específicas:

- **Funcionalidad** capacidad del software para realizar las tareas y funciones para las cuales fue diseñado.
- **Fiabilidad,** capacidad del software de funcionar de manera confiable y consistente en diferentes situaciones y entornos.
- **Usabilidad**, facilidad de uso y comprensión del software por parte de los usuarios. Un software con buena usabilidad es intuitivo, cuenta con una interfaz clara y

amigable, y permite a los usuarios realizar sus tareas de manera eficiente y sin dificultades innecesarias.

- Se divide en sub características como:
- La comprensibilidad,
- La apariencia estética,
- La facilidad de aprendizaje.

- **Eficiencia**, se refiere al rendimiento y la capacidad de respuesta del software en términos de velocidad, consumo de recursos y tiempo de ejecución.
- **Mantenibilidad**, se refiere a la facilidad y la capacidad de realizar modificaciones y mejoras en el software.
- **Portabilidad,** se refiere a la capacidad del software de funcionar correctamente en diferentes plataformas y entornos.

- La norma también proporciona métricas que se utilizan para medir y cuantificar la calidad del software en función de los criterios establecidos, las cuales puede ser cualitativas o cuantitativas,
- Contribución a la Confidencialidad de Datos Médicos: En versiones completas de ISO 25000, la norma en general abarca la seguridad como una característica crítica de calidad.

ISO/IEC 25000 (SQuaRE): Esta familia de normas se enfoca en la "Calidad del Producto Software", definiendo un modelo de calidad con características principales y subcaracterísticas para evaluar el software. Al exigir un software "seguro", ISO/IEC 25000

directamente apoya la confidencialidad, Un software seguro es un software que protege la información del acceso no autorizado.

En la gestión de acceso y autenticación, asegurar que solo el personal autorizado acceda a los datos, y proteger la privacidad, la información personal del paciente. Las acciones realizadas deben quedar registradas como una prescripción médica o un diagnóstico previo.

- La norma principal de esta familia es la ISO/IEC 25000:2014, que se titula "Sistemas y software de ingeniería de sistemas de tecnología de la información - Calidad del producto".
- La norma ISO/IEC 12207 (2008) **(Ciclo de Vida del Software):** De acuerdo con Piattini (2019), es un estándar internacional que establece los procesos y actividades necesarios para el ciclo de vida del software, esta norma proporciona un marco completo y sistemático para el desarrollo, adquisición, suministro, operación y mantenimiento de sistemas y productos de software.
- Al definir procesos sistemáticos como la **gestión de requisitos**, el **diseño**, la **implementación** y las **pruebas**, ISO/IEC 12207 asegura que los requisitos de precisión (establecidos por el dominio de salud y por normas como ISO 25000) sean capturados, implementados y verificados en cada etapa, por ejemplo, un proceso de pruebas riguroso (definido en 12207) es fundamental para validar la exactitud de los cálculos y la integridad de los datos.
- La "gestión de configuración" asegura que solo versiones autorizadas del software y los datos se manejen. Al estandarizar estos procesos, 12207 crea una metodología robusta que incorpora la confidencialidad desde la concepción hasta la operación.

- La norma ISO/IEC 12207 define los procesos fundamentales que deben llevarse a cabo en cada etapa del ciclo de vida del software, desde la identificación de los requisitos hasta la entrega y el mantenimiento del producto final. Estos procesos incluyen la gestión de requisitos, el diseño, la implementación, las pruebas, la gestión de configuración y la documentación, se divide en:
- **Proceso Principales**
- **Procesos de Soporte**
- **Procesos de Organización**
- La norma ISO/IEC 15504 define un modelo de referencia que se compone de cinco niveles de madurez, donde cada nivel representa un grado creciente de madurez y capacidad en los procesos (Piattini, 2018):
- **Nivel 1: Proceso inicial (Incomplete)**
- **Nivel 2: Proceso administrado (Performed)**
- **Nivel 3: Proceso definido (Established)**
- **Nivel 4: Proceso gestionado cuantitativamente (Predictable)**
- **Nivel 5: Proceso optimizado (Optimizing)**
- Cada nivel de madurez en la norma ISO/IEC 15504 representa un avance en la capacidad y la calidad de los procesos de software en una organización, proporcionando un camino claro para la mejora continua y el logro de resultados consistentes y de alta calidad.

3.- Comprender los Requisitos del Cliente:

- **Especificación Clara:** Traducir las normativas (como ISO/IEC 25000 y 12207) en requisitos funcionales y no funcionales explícitos para el software como por

ejemplo, el cliente requiere que el sistema debe cifrar los datos de paciente en tránsito y en reposo según el estándar definido, el sistema debe registrar todas las operaciones de acceso a la historia clínica con fecha, hora y usuario, asegurando la calidad de los datos mediante verificaciones al ingreso y salida de los datos, con las alarmas de aviso requeridas.

- Para comprender los requisitos del cliente el equipo de QA debe participar desde esta fase para asegurar que los requisitos sean verificables y que las normativas sean consideradas desde el diseño, para ser medibles y auditables, realizando la documentación pertinente.

4.- Investigar las Mejores Prácticas de la Industria: El CMMI Capability Maturity Model Integration, es un modelo para la mejora y evaluación de procesos en áreas como el desarrollo de software, la gestión de servicios y la adquisición, ayuda a las organizaciones a establecer prácticas efectivas de gestión y desarrollo de software, mejorando la calidad, la eficiencia y la capacidad de respuesta a los requisitos del cliente.

- Considerar aspectos como la gestión de la seguridad de los datos.
- El acceso.
- La autenticación.
- La auditoría.
- La gestión de riesgos para garantizar el cumplimiento de las normativas y la calidad del software desarrollado.
- Ejecutar pruebas.

5.- Idear un Plan Integrado:

- **Plan de Pruebas:** Describe el alcance, los objetivos, los recursos, el cronograma y la estrategia de pruebas, incluyendo cómo se abordarán las normativas.
- **Casos de Prueba:** Especificaciones detalladas de cómo se validará cada requisito y norma.
- **Reportes de Defectos:** Documentación de cualquier desviación encontrada, su impacto y su seguimiento.
- **Resultados de Pruebas:** Registros de la ejecución de pruebas y la evidencia de cumplimiento/incumplimiento.
- **Informes de Resumen de Pruebas:** Documentos finales que evalúan la calidad del software y el nivel de cumplimiento de las normativas.

- Recopilar evidencia y documentar los resultados para demostrar el cumplimiento.
- La documentación es crucial y debe ser continua.

6.- Es fundamental operar por etapas.

Esto se conoce como el **Ciclo de Vida de las Pruebas de Software (STLC - Software Testing Life Cycle)**, que se alinea con el Ciclo de Vida del Desarrollo de Software (SDLC).

Etapas comunes del STLC:

- **Análisis de Requisitos de Prueba:** Comprender los requisitos funcionales y no funcionales (incluyendo las normas de calidad, seguridad de los datos, privacidad) para identificar qué se debe probar, como los fragmentos de cada función(pruebas unitarias), codificación estándar, conexión con la base de datos, funcionamiento de rutas, que no exista repetición de código, las querys

deben estar escritas preparadas para que no haya inyección de código sql, que el sistema no se caiga, etc.

- **Planificación de Pruebas:** Definir el alcance, los objetivos, la estrategia, el tipo de pruebas, los recursos y el cronograma(aplicación jira), como existen segmentos de código que son prerrequisito de otros, entonces las pruebas deben ser consecuentes con el cronograma, siempre con las normativas en mente. **Aquí es donde las normas y las metas del proyecto se traducen en acciones concretas.**
- **Diseño de Casos de Prueba:** Crear los casos de prueba detallados a partir de los requisitos y la estrategia, dependiendo del proyecto se pueden hacer pruebas manuales, automatizadas, programas de acción en terreno físicas, ejemplo con equipos puesta en marcha de un servidor.
- **Configuración del Entorno de Pruebas:** Preparar el hardware y software necesarios para ejecutar las pruebas.
- **Ejecución de Pruebas:** Realizar las pruebas según los casos de prueba diseñados, registrando resultados y defectos.
- **Cierre del Ciclo de Pruebas:** Evaluar los criterios de salida, generar informes finales, y documentar las lecciones aprendidas para futuros proyectos.
- **Cumplimiento de Metas:** La QA no solo encuentra errores; asegura que el producto cumpla con los **criterios de calidad** y los **objetivos del proyecto**

7.- Organizar las Pruebas: Es un plan con pasos definidos y sistemáticos, como un algoritmo o una metodología. No es solo una declaración de intenciones. Los marcos como el STLC, o incluso metodologías más amplias como CMMI o los principios de Agile/Scrum, proporcionan esta estructura, la función es difundirlas y aplicarlas.

La incorporación en la Fase de Diseño, Diseño de Seguridad y Privacidad por Defecto (Security & Privacy by Design) y asegurarse de que la arquitectura del software incluya componentes y mecanismos específicos para cumplir con los requisitos de seguridad, privacidad y confidencialidad (ej., módulos de autenticación robusta, control de acceso basado en roles, anonimización de datos).

- **Modelado de Amenazas:** Identificar y mitigar posibles amenazas a la seguridad y privacidad en la fase de diseño.
- **Aplicación en la Fase de Implementación/Codificación:** El código debe ser reutilizable, ordenado, entendible, funcional.
- **Estándares de Codificación Segura:** Obligar a los desarrolladores a seguir guías de codificación que prevengan vulnerabilidades comunes (ej., OWASP Top 10).
- **Revisiones de Código:** Realizar revisiones de código enfocadas en la seguridad, el cumplimiento de requisitos específicos de las normativas y funciones del algoritmo.
- **Herramientas de Análisis Estático (SAST):** Utilizar herramientas que analicen el código fuente automáticamente en busca de vulnerabilidades.
- **Verificación y Validación Rigurosa (Fase de Pruebas):** Pruebas unitarias, de integración.
- **Plan de Pruebas Detallado:** El plan de pruebas creado debe conocerse por el personal, capacitar para el logro del cumplimiento exhaustivo de las pruebas es parte del éxito de estas, aunque parezca riguroso es imprescindible para realizar mediciones correctas sin sesgos, en un plan de pruebas que mapee explícitamente los casos de prueba a los requisitos normativos.

Tipos de Pruebas Específicos:

- **Pruebas de Funcionalidad y Exactitud:** Para la precisión de los datos (basado en ISO/IEC 25000).
- **Pruebas de Seguridad:** Pruebas de penetración, escaneo de vulnerabilidades, pruebas de autenticación/autorización.
- **Pruebas de Privacidad:** Para asegurar el manejo correcto de datos sensibles.
- **Pruebas de Fiabilidad y Rendimiento:** Para asegurar la estabilidad y capacidad de respuesta.
- **Pruebas de Regresión:** Para asegurar que los cambios no introduzcan nuevas fallas o rompan el cumplimiento existente.
- **Automatización de Pruebas:** Automatizar pruebas repetitivas para asegurar una cobertura consistente y eficiente.
- **Pruebas unificadas y Pruebas integradas:** Comienzan a realizarse las pruebas de sistema.

8.- Documentar Continuamente:

Documentación de Resultados de Pruebas:

Registrar meticulosamente los resultados de cada prueba, incluyendo evidencia de cumplimiento o defectos.

Gestión de la Configuración y Versiones:

Utilizar sistemas de control de versiones para todo el código, la documentación y los artefactos de prueba, asegurando la trazabilidad y el control sobre los cambios.

Documentación y Trazabilidad Exhaustivas:

Mantener un registro completo de todos los requisitos, diseños, pruebas, defectos y cambios. Esto es fundamental para la auditabilidad y para demostrar el cumplimiento ante entidades reguladoras.

Auditorías Internas y Externas:

Realizar auditorías internas periódicas para evaluar el cumplimiento de los procesos y las normativas. Prepararse para auditorías externas por parte de organismos reguladores.

Capacitación Continua del Equipo:

Asegurarse de que todo el equipo de desarrollo, QA y operaciones esté bien informado sobre las normativas relevantes, las políticas de la empresa y las mejores prácticas de seguridad y privacidad, no olvidar informar de los cambios y mejoras en las implementaciones del sistema.

Dentro de las responsabilidades de un profesional QA puedo inferir que las visualizaciones objetivas para ver y compartir avances deben permanecer fieles a los datos y avances incluso mantenerse dispuestos a ser presentados a los clientes.

Dentro de estas responsabilidades están establecer un proceso formal para:

- Identificar,
- Analizar,
- Mitigar,
- Monitorear los riesgos relacionados con la calidad, seguridad y cumplimiento normativo del software.

La Mejora Continua del Proceso (ISO/IEC 15504, CMMI):

- Evaluar periódicamente la madurez de los procesos de desarrollo y QA utilizando modelos como ISO/IEC 15504 (que define niveles de madurez) o CMMI.

Según como menciona Báez Pérez, C. I. y Suárez Zarabanda, M. I. (2013). Los estándares, traducidos como métricas, se han concentrado en puntos claves que garantizan el producto evaluado y le brindan al usuario las coordenadas necesarias para escoger aquel que se ajusta a sus supuestos.

Éstas métricas son4:

- Métricas del software.
- Funcionalidad,
- Complejidad,
- Eficiencia.

Se relacionan con el desarrollo mismo.

- Métricas técnicas. Al centrarse en las características del software (complejidad lógica, grado de modularidad) miden su estructura y evalúan cómo está hecho.
- Métricas de calidad. Indican cómo se ajusta a los requisitos implícitos y explícitos del cliente.
- Métricas de productividad. Se enfocan en el rendimiento del proceso de ingeniería. Por consiguiente, indican qué tan productivo puede llegar a ser el software a diseñar.
- Métricas orientadas a la persona. Se aplican al personal que desarrolla el sistema. Proporcionan información, especialmente desde el punto de vista humano, concerniente a la efectividad de las herramientas y metodologías utilizadas.

- Métricas orientadas al tamaño. Se asocian con el proceso de producción. Permiten establecer el tiempo y el número de personas que se requieren para terminar el software.
- Métricas orientadas a la función. Estas medidas indirectas señalan la funcionalidad o utilidad del programa.
- **CMMI-SW** se enfoca en la mejora de los procesos de software.
- **IPD-CMM** extiende esa idea a todo el ciclo de vida del desarrollo de productos integrados (hardware + software + otros).
- **EIA/IS** representa estándares de la industria electrónica que a menudo tocan los procesos de desarrollo de productos y software dentro de ese sector.

CATEGORÍAS DEL CMMI, trabaja con cuatro categorías:

- Ingeniería de sistema. Cubre la construcción de un sistema con o sin software.
- Ingeniería de software. Abarca la producción de soluciones software.
- Integración de productos y procesos de desarrollo. Comprende la relación a largo plazo con el cliente.
- Relación con proveedores. Se refiere a las dinámicas relacionadas con la subcontratación de partes para el sistema.

Báez Pérez, C. I. y Suárez Zarabanda, M. I. (2013). Proceso de desarrollo de software: basado en la articulación de RUP y CMMI priorizando su calidad: (ed.).

Figura 2. Áreas de proceso CMMI

Puedo observar que en cuanto a las normas y las necesidades de los clientes podemos conseguir como programadores los estándares que pueden definir los algoritmos, las pruebas pueden corregir nuestro algoritmo, pero la trazabilidad de las reglas que podríamos utilizar como métricas que orienten nuestro trabajo, sin duda marcan una pauta que se presenta muy efectiva, para el desarrollo no tan solo de cada parte del software sino también de todos sus complementos en el momento de la integración.

Al ver este proceso me resulta fundamental la rigurosa implementación y la tranquilidad de saber que a medida que se avanza, cada paso ha sido probado, impidiendo la incertidumbre de la validación final. En tanto las empresas deben dar un enfoque más serio a el proceso de QA ya que los estudios y el análisis demuestran que la certificación de calidad de una empresa y su madurez en el proceso del sistema es muy valorada por la industria y los consumidores debido a la calidad en los resultados, identificando la importancia del QA y las certificaciones existentes, tanto para empresas como para

profesionales, por ejemplo, **ISTQB® Certified Tester Foundation Level 4.0 (CTFL), para profesionales.**

REFERENCIAS BIBLIOGRÁFICAS

https://iso25000.com/index.php/normas-iso-25000

Báez Pérez, C. I. y Suárez Zarabanda, M. I. (2013). Proceso de desarrollo de software: basado en la articulación de RUP y CMMI priorizando su calidad: (ed.). Tunja, Universidad de Boyacá. Recuperado de https://elibro.net/es/ereader/iacc/129062?page=15.

https://www.ispch.gob.cl/wp-content/uploads/2021/02/Politica_softwares_v1.pdf

https://www.bcn.cl/leychile/navegar?idNorma=1190336

https://rise.articulate.com/share/exurEaTbFpmup-CNxwl-QUYqByTjtQeZ#/lessons/OU396mDclQUACUF84wMsA2jcyJ83kDm